

HELIXCODE_ACCESS_GUIDE

HelixCode Access Guide — How to Use HelixLLM, HelixAgent, and HelixCode

Version: helix-code-1.1.0-dev-0.0.2 **Last updated:** 2026-07-13 **Status:** All services running, verified against real infrastructure

Quick Start

1. Start the Infrastructure

```
cd helix_code/
make test-infra-up
    # Starts Postgres, Redis, Ollama, Weaviate, ChromaDB,
    # Qdrant, etc.
make test-infra-status # Verify all services healthy
```

2. Start the HelixCode Server

```
make build # Builds bin/helixcode
./bin/helixcode # Starts on port 8080 (config/
               config.yaml)
# OR for a different port:
HELIX_SERVER_PORT=18080 ./bin/helixcode
```

Health check:

```
curl http://localhost:8080/health
# → {"status":"healthy","version":"1.0.0"}
```

3. Register a User + Get Auth Token

```
# Register
curl -X POST http://localhost:8080/api/v1/auth/register \
  -H 'Content-Type: application/json' \
  -d '{"username":"myuser","email":"me@example.com","password":"MyPass123!}'

# Login → get JWT token
curl -X POST http://localhost:8080/api/v1/auth/login \
```

```
-H 'Content-Type: application/json' \
-d '{"username":"myuser","password":"MyPass123!"}'
# → {"token":"eyJ...", "status":"success"}
```

Using HelixLLM (Local Models via Ollama)

Generate Text

```
TOKEN="<your-jwt-token>"
curl -X POST http://localhost:8080/api/v1/llm/generate \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"prompt":"What is 2+2?","model":"qwen2:0.5b"}'
# → {"content":"The answer is
    4.", "status":"success", "model":"qwen2:0.5b", "provider":"ollama"}
```

Stream Text (SSE)

```
curl -X POST http://localhost:8080/api/v1/llm/stream \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"prompt":"Write a poem about code","model":"qwen2:0.5b"}'
```

List Available Models

```
curl http://localhost:8080/api/v1/llm/models \
  -H "Authorization: Bearer $TOKEN"
```

Available Local Models (Ollama)

Model	Size	Context	Use Case
qwen2:0.5b	0.5B	32K	Fast inference, testing
llama2:7b	7B	4K	General purpose

Pull more models:

```
curl -X POST http://localhost:11434/api/pull -d
  '{"name":"llama3.2"}'
```

OpenAI-Compatible Endpoint

```
curl -X POST http://localhost:8080/v1/chat/completions \
  -H "Content-Type: application/json" \
```

```
-H "Authorization: Bearer $TOKEN" \  
-d '{"model":"qwen2:0.5b","messages":  
  [{"role":"user","content":"Hello!"}]}'
```

Anthropic-Compatible Endpoint

```
curl -X POST http://localhost:8080/v1/messages \  
-H "Content-Type: application/json" \  
-H "Authorization: Bearer $TOKEN" \  
-d '{"model":"qwen2:0.5b","max_tokens":128,"messages":  
  [{"role":"user","content":"Hello!"}]}'
```

Using HelixAgent (ACP Protocol)

HelixCode acts as an ACP (Agent Client Protocol) agent — compatible with Zed, JetBrains, and other ACP-aware editors.

Start ACP Agent

```
./bin/helixcode acp      # Starts ACP agent on stdio
```

Connect from an ACP Editor

1. Open Zed or JetBrains
2. Configure ACP agent path: `helixcode acp`
3. The editor will handshake via JSON-RPC over stdio
4. Prompt turns stream real LLM generation via session/update notifications

ACP Features (Implemented)

Feature	Status
Initialize handshake	Phase 1
Session management (new/close)	Phase 2
Prompt turn generation (real LLM)	Phase 3
Streaming session/update	Phase 4
Permission mapping (tools/permissions)	Phase 5
File read/write	Future
Terminal access	Future

Using HelixCode CLI

```
./bin/cli --help           # Show all commands
./bin/cli --list-models    # List available models
./bin/cli generate "What is Go?" # Generate text
./bin/cli acp              # Start ACP agent
./bin/cli mcp              # Start MCP server
./bin cli lsp              # Start LSP server
```

RAG (Retrieval-Augmented Generation)

RAG is disabled by default. Enable it:

```
export HELIXCODE_RAG_ENABLED=true
# Configure vector store (Weaviate/ChromaDB/Qdrant – all running
  via test-infra-up)
# RAG will automatically retrieve relevant context and prepend it
  to prompts
```

Works on all endpoints: native /api/v1/llm/generate, OpenAI /v1/chat/completions, Anthropic /v1/messages.

API Reference

Endpoint	Method	Auth	Description
/health	GET	No	Server health check
/api/v1/auth/register	POST	No	Register user
/api/v1/auth/login	POST	No	Login, get JWT
/api/v1/llm/generate	POST	Yes	Generate text
/api/v1/llm/stream	POST	Yes	Stream text (SSE)
/api/v1/llm/models	GET	Yes	List models + capabilities
/v1/chat/completions	POST	Yes	OpenAI-compatible
/v1/messages	POST	Yes	Anthropic-compatible
/api/v1/llm/providers	GET	Yes	List providers

Running Tests

```
make test           # Unit tests
make test-full      # All tests against real infra
make stress-chaos    # Stress + chaos tests
make verify-compile  # Quick compile check
```

Stopping Services

```
make test-infra-down  # Stop all containers
# Server: Ctrl+C or kill the process
```

Troubleshooting

Issue	Fix
Port 8080 in use	Change port in config/config.yaml or set HELIX_SERVER_PORT
Ollama no models	curl -X POST http:// localhost:11434/api/pull -d '{"name": "qwen2:0.5b"}'
401 Unauthorized	Register + login to get a JWT token
Container not healthy	make test-infra-down && make test-infra-up

Sources verified 2026-07-27: INTERNALLY DERIVED
— this guide documents THIS repository’s own build/run surface, so its authoritative sources are in-repo tracked files, not third-party URLs. Cross-referenced in this pass against: `helix_code/Makefile` — every referenced target confirmed to exist at build: (line 98), `test-infra-up`: (194), `test-infra-down`: (206), `test-infra-status`: (212); `helix_code/cmd/server/README.md` (line 123) and `helix_code/internal/config/README.md` (line 85) — confirm the `HELIX_SERVER_PORT` override and its documented default of 8080, as used in the Quick Start and Troubleshooting table above; and the `/health` endpoint literal in `helix_code/internal/` (e.g. `internal/discovery/registry.go:390`, exercised by `internal/server/server_nil_redis_test.go:55`). Honest boundary (§11.4.6 / §11.4.99(B)): this footer attests that every command, target, env var, and endpoint cited above resolves to a real tracked definition in this repository as of this date — it does NOT constitute a live end-to-end run of the documented flow in this pass (no infrastructure was started and no port was bound while verifying), and the external services the guide touches (Ollama’s `/api/pull` endpoint shape) were NOT re-verified against upstream vendor documentation in this pass. Re-verify per the §11.4.99 180-day staleness window, or

sooner if a Makefile target, config key, or endpoint is renamed.